

Dubljy project in the first phase

To facilitate the workflow and ensure an organized development process, the first phase of our project was broken down into a series of clear and manageable steps. Below is a summary of the work completed so far:

1. Video Analysis and Title Extraction:

We began by identifying and processing the video input. The title of the video was automatically retrieved to aid in organizing and labeling files throughout the project.

2. Video Download (MP4 Format):

The video was downloaded in MP4 format to make it easier to handle during the dubbing and merging stages. This format ensures compatibility with most editing and processing tools.

3. Audio Extraction (MP3 Format):

The audio was then separated from the video and saved as an MP3 file. This step helps reduce file size and simplifies the audio processing tasks that follow.

4. Audio-to-Text Conversion with Timestamps:

The extracted audio was converted into text with high time accuracy. This is a key step for aligning translated or dubbed content with the original video, and it prepares the material for seamless integration in the next stages.

5. API Integration and Project Linking:

We've also begun the process of API development and integration, which will serve as the core mechanism for connecting the different parts of the system and automating the workflow.

Step 1: Video Analysis and Title Extraction:

Once the user enters the video link, the system retrieves the video address and verifies that it matches the intended content. One of the key requirements we set early on was that the video source must be from **YouTube (for this time)**, as it is the primary platform students rely on for educational content.

To determine the most effective tool for retrieving and handling YouTube videos, we tested multiple models. Below is a summary of our evaluation:

1. Pytube

- **Performance:** Slow to retrieve video data and often inaccurate. We experienced several errors while fetching different videos.
- **Limitations:** It only supports YouTube, which limits future scalability. It also does not automatically merge audio and video streams into high-quality outputs.
- **Technical Challenge:** Requires manual handling with tools like `ffmpeg` to combine audio and video, which complicates the process and increases the risk of user error.
- **Status: Not approved** for our project due to performance and usability concerns.
- `pip install pytube` && Download from: <https://ffmpeg.org/download.html>

2. Yt-dlp

- **Performance:** Fast, accurate, and highly efficient in retrieving video content with minimal error.
- **Compatibility:** Supports a wide range of video sources, including YouTube and additional metadata resources.
- **Integration:** Seamlessly links with `ffmpeg`, allowing for automated extraction and high-quality merging of audio and video streams.
- **Usability:** Simple and effective to use, minimizing manual effort.
- **Status: Approved** for our DublJly project. Its speed, reliability, and integration capabilities make it the ideal choice for our workflow.
- `pip install yt-dlp`

Model	Advantages	Disadvantages	Status
Pytube	– Supports YouTube – Basic functionality for downloading	– Slow and inaccurate address retrieval – Frequent errors with multiple videos – Only works with YouTube (limited scalability) – Does not merge video/audio automatically – Requires manual integration with <code>ffmpeg</code>	Not Approved

Yt-dlp	– Fast and accurate – Very low error rate – Supports YouTube and other metadata sources – Automatically linked with ffmpeg for seamless merging – High usability and efficiency	– None significant noted during testing	Approved for the Dubljly Project
--------	---	---	--

Step 2: Video Download (MP4 Format):

At this stage, our goal was to download the video in MP4 format in order to use it later in the merging process with the dubbed audio. Since the model (yt-dlp) we used to retrieve the video title also supports automatic separation of video and audio, we decided to reuse it for efficiency and consistency.

We configured the system to save the downloaded video to a fixed directory, making it easier to retrieve and reuse in the following steps of the workflow. This also helps maintain a structured and organized file system throughout the project.

Step 3: Audio Extraction (MP3 Format)

To simplify the audio-to-text conversion process, we extracted the audio directly from the video file. Working with full video files tends to be more cumbersome, consumes more storage space, and can reduce processing efficiency.

After experimentation, we found that converting the audio to MP3 format significantly improved performance and reduced memory usage. As in previous steps, we used the same model that supports automatic separation of video and audio streams. The extracted audio was saved in a dedicated file to allow for easy and organized retrieval in the next processing stage.

Step 4 :Audio-to-Text Conversion with Timestamps:

At this stage, we began the process of **transcribing the extracted audio into text**, taking into account the **actual timestamps** for accurate alignment during the final video merging phase. Our primary focus was on:

- **Accuracy of speech recognition**
- **Precision in time tracking**
- **Elimination of external noise** to ensure clarity and clean transcription

The output text was saved in a `.txt` file, organized and ready to be used in the next stages.

To achieve the desired quality, we tested several transcription models. However, we encountered various issues that affected performance and efficiency. These included:

- Inaccurate timing (e.g., fixed intervals like 30 seconds regardless of speech)
- Improper handling of pauses or breaths
- Low accuracy in converting spoken content to text
- Time-consuming processing speeds

Due to these limitations, we carefully evaluated multiple models, which we will now present.

Google Speech-to-Text:

- Requires internet connection
- Pricing based on usage (potentially expensive for high volume)
- It is highly efficient and prevents disturbance, but because it is paid, we have excluded its use.
- Not approved

Coqui:

- It is lightweight, fast, and easy to train, making it suitable for embedded systems or custom pipelines.
- Its accuracy is significantly lower than that of modern models like Whisper or Google STT, especially in noisy environments or for languages with complex phonetics.
- supports offline use.
- Lacks smooth handling of punctuation, pauses, and time marks at the word level.
- The model's architecture does not have real-time extraction technology, which needs to be added, which complicates the matter.
- Not approved

Wav2Vec 2.0:

- Does not include native timestamps.
- It can be fine-tuned for very high accuracy on specific languages or domains (especially English)
- it supports offline processing
- does not provide automatic punctuation
- does not provide multilingual output
- Not approved

OpenAI Whisper:

- It handles a wide range of accents, background noise, and languages very well.
- Its strength lies in accuracy and generalization
- Whisper processes audio in fixed 30-second chunks, which can lead to unnatural cuts, incorrect segmentation, and missed pauses or speaker changes.
- Need high GPU or CPU
- Not approved

FastWhisper:

FastWhisper is a **highly optimized reimplementation of Whisper**, running via **ONNX** for significantly faster inference. It maintains all of Whisper's strengths (multilingual support, high accuracy, robustness to noise), but fixes some of Whisper's key limitations:

- **Pause-aware segmentation:** It naturally detects pauses and silence, splitting transcripts accordingly. This solves the major issue in Whisper where audio was cut into 30-second blocks regardless of sentence structure or silence.
- **Efficient performance:** Thanks to ONNX runtime, it runs **much faster**, especially on CPUs or edge devices.
- **Timestamps and word-level granularity:** It still supports precise timestamps, making it suitable for subtitling or transcript alignment.
- **Fully offline:** No cloud access needed.
- **Same model sizes** as Whisper (tiny → large), allowing scalability depending on your resource availability
- **Approved**

Model	Key Features	Limitations	Approval Status
Google STT	- Highly accurate - Handles accents & noise well - Real-time streaming	- Requires internet - Paid API - Not suitable for high-volume projects	Not Approved
Coqui STT	- Lightweight and fast - Easy to train and customize - Works offline	- Low accuracy in noisy/complex settings - No proper punctuation or timestamps - No real-time sync	Not Approved
Wav2Vec 2.0	- Offline use - High accuracy if fine-tuned - Good for English	- No timestamps - No punctuation - No multilingual support	Not Approved
OpenAI Whisper	- High accuracy - Multilingual support - Strong noise tolerance	- Needs powerful hardware - 30s chunk processing causes segmentation issues	Not Approved
FastWhisper	- Pause-aware segmentation - Timestamp and word-level alignment - Fast & efficient with ONNX - Fully offline - Scalable model sizes	- Requires ONNX setup (minor)	Approved

Next step → API with Django